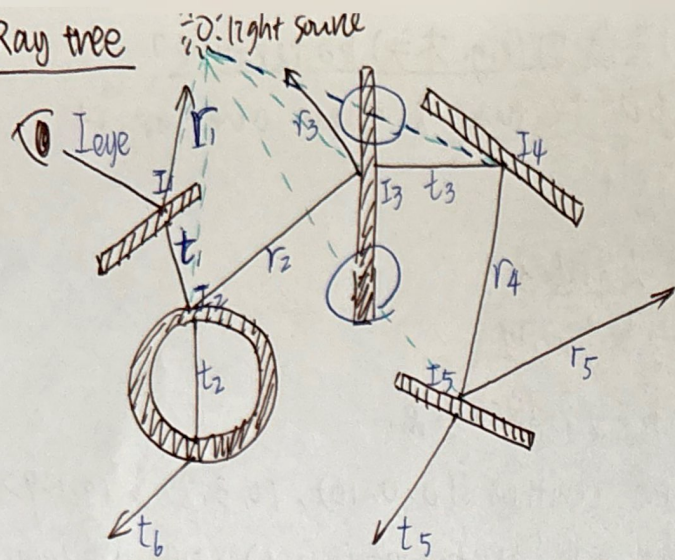
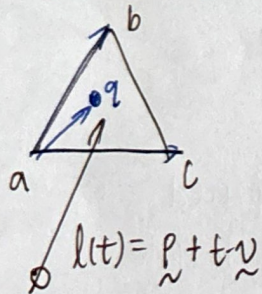


• Ray tree



checking the shadow ray,
发现 I4 & I5 都被挡住
→ ∴ I4 & I5 只有 ambient term.
而只有 I1, I2, I3 才是完整的 PIE.

• 光线与△相交判断条件.



△内任意一点 q 可表示为:

$$q = a + \alpha(b-a) + \beta(c-a),$$

where $\alpha, \beta \geq 0$ and $\alpha + \beta \leq 1$
if $t \geq 0$ and $\alpha, \beta \geq 0$
and $\alpha + \beta \leq 1$, 相交!

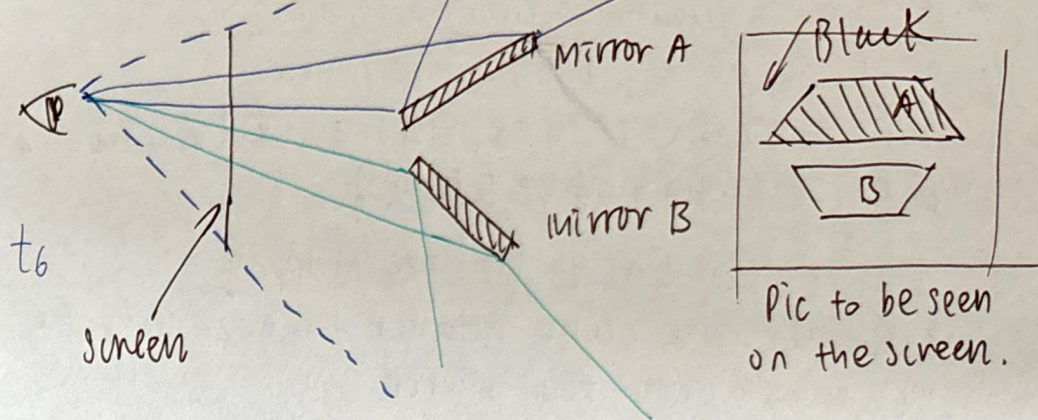
∴ if $l(t)$ intersects abc , $q = l(t) = p + t*u$

$$\Rightarrow p + t*u = a + \alpha(b-a) + \beta(c-a)$$

∴ 代入化简:

$$\begin{bmatrix} \alpha \\ \beta \\ t \end{bmatrix} = \begin{bmatrix} b_x - a_x & c_x - a_x & u_x \\ b_y - a_y & c_y - a_y & u_y \\ b_z - a_z & c_z - a_z & u_z \end{bmatrix}^{-1} \begin{bmatrix} p_x - a_x \\ p_y - a_y \\ p_z - a_z \end{bmatrix}$$

• # of Ray tracing with respect to mirrors



Grey: $2 \times \# \text{灰} + 1 \times \# \text{白}$

White: $2 \times \# \text{白} + 1 \times \# \text{灰}$

Black: $2 \times (\# \text{全} - \# \text{灰} - \# \text{白})$

total.

* # of tests increases exponentially with # objects.

• Mesh simplification

Pros	Cons/ Trade-off
Faster Rendering performance	Loss of surface detail/ accuracy
Smaller file size	Possible <u>shape distortion</u> if simplification is too aggressive
Efficient transmission & storage	Texture or edge sharpness may degrade.
Enables multiple LOD models.	Requires careful algorithm tuning

• Mapping Techniques (续)

→ 如何生成 "Accurate" Mirror reflection?

① Duplicate an obj behind the mirror
then 正常渲染这个 "背后的复制体", 使其看上去就好像是真实物体在镜子里被反射出来。

② planar reflection + texture mapping

→ Render the scene another time & map the "picture" onto the mirror

即, 一张从 "镜中视角" 渲染出来的图, 被贴在 "镜子" 这块多边形上。

③ Other: cube map / ray trace small & reflective obj.

Cube map: 对于小的、近似球状的反射物体, 常用 environmental mapping → 一次生成, 多应用

Ray tracing: 对于很小、局部的高光反射细节 / 在现代 GPU (支持硬件 ray tracing) 上, 对小的反射物体做简化版 ray tracing.

→ 如何用 mapping 来生成 阴影? (比较真实的阴影)

• 从光源的视角 render 场景, 得到一张 shadow map (a depth texture) → 记录了 "最靠近光的那层表面" 到光的距离。

• 在 clip / texture space 中, 用 (u, v) 去查这条光线上的最近表面深度 h . if $h \leq \text{shadowMap}(u, v) \rightarrow$ 被照亮
else if $h > \text{shadowMap}(u, v) \rightarrow$ 阴影。

→ 如何生成物体横截面 (e.g. 木头) 的 texture?

• Construct a 3D texture & then overlap it with the object.

• 光线与哪个物体先相交?

[Step 1. 求光线的参数方程.

Step 2. 代入物体的方程求交点.

e.g. 三个球 with centres $(0, 0, 10)$, $(0, 3, 8)$ & $(0, 1, 9)$, 半径均为 2 units. ray: centre at $(0, 0, 0)$ with $\vec{d} = (0, 0, 1)$

$$l(t) = (0, 0, 0) + t(0, 0, 1)$$

$$= (0, 0, t)$$

$$\text{球①: } (x-0)^2 + (y-0)^2 + (z-10)^2 = 2^2 = 4$$

$$\text{代入 } t: 0 + 0 + (t-10)^2 = 4$$

$$\Rightarrow t = 8 \text{ or } 12.$$

$$\text{球②: } (x-0)^2 + (y-3)^2 + (z-8)^2 = 2^2 = 4$$

$$\text{代入 } t: 0 + 9 + (t-8)^2 = 4$$

$$\Rightarrow t = \text{undefined}$$

$\Rightarrow \therefore$ ray does not intersect with sphere 2.

$$\text{球③: } (x-0)^2 + (y-1)^2 + (z-9)^2 = 2^2$$

$$\text{代入 } t: 0 + 1 + (t-9)^2 = 4$$

$$\Rightarrow t = 10.73 \text{ or } 7.27$$

$$\therefore 7.27 < 8 < 10.73 < 12$$

$\therefore$ The ray first intersects 球③, 交点为 $l(7.27) = \begin{pmatrix} 0 \\ 0 \\ 7.27 \end{pmatrix}$